

Library Management System

B100C Python Programming

George Topuria

GH1045086

Github:

<https://github.com/georgetopuria/George-Topuria-Library-Management-System>

1 Introduction

1.1 Background

Libraries need to deal with numerous books, patrons, and borrowing and returning records. Usually these activities are done manually on paper and in spreadsheets. However, manual management is very inefficient and prone to mistakes and duplicate records, loss of information, or problems with books being borrowed.

Computer programming offers an efficient means of automating these processes. A software system can be created to store information electronically, retrieve records in a timely fashion and minimize human error. In this project, the practical problem of a business is solved using Python programming by creating a Library Management System.

1.2 Aim of the Project

This project is aimed at designing and developing a Library Management System in Python. The system enables users to control books, members and borrowing activities with an easy-to-use menu-driven interface.

The project was designed to implement the key concepts learnt in the module such as Object Oriented Programming, Functions, File handling, Exception handling, Control structures and Modular program design.

1.3 Scope

The system offers the following functions:

Add new books to the library.

Add new members.

View existing books.

View registered members.

Borrow books.

Return books.

View loan records.

Write data to CSV files.

Automatically load previously saved data.

Do not allow duplicate book and member IDs.

The project is designed for a small library setting and will be limited to the essential functions of a library, not to the more sophisticated features like graphical interfaces or online databases.

2 System Design

2.1 Overview

The system is designed in an object oriented design approach. Object-oriented programming enables related data and behaviours to be bundled into classes. This improves code organization, readability and maintainability.

The project consists of four main classes:

Book

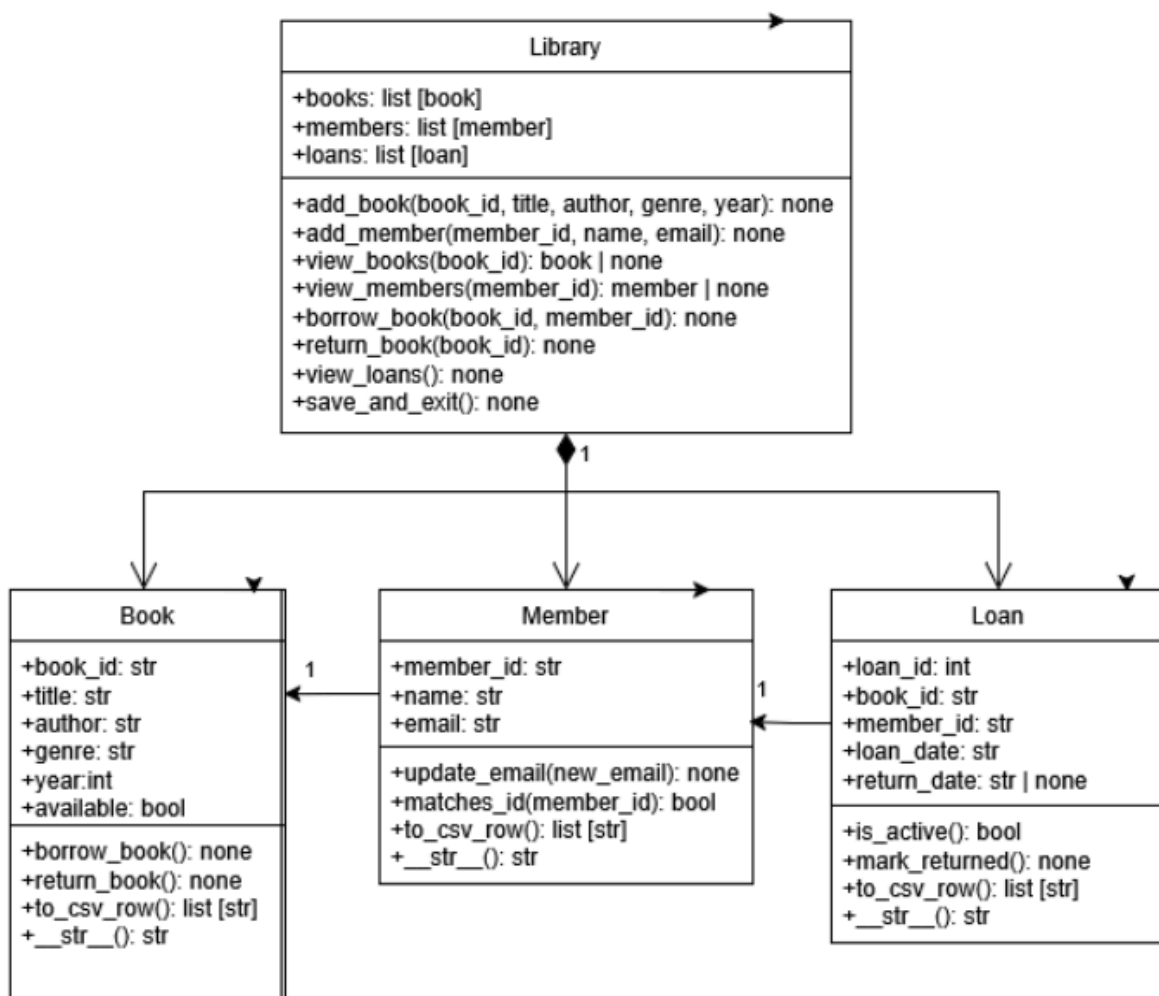
Member

Loan

Library

Each class is responsible for a specific aspect of the system.

A UML class diagram of the library management system:



The relationships between the four classes used in the application are shown in figure above. The Library class is the main controller, the Book, Member, and Loan are the core tiles that are controlled by the system.

2.2 Book Class

The Book class is a representation of a single book in the library.

Attributes include:

Book ID

Title

Author

Genre

Publication Year

Availability Status

The class provides the following methods: Borrow a book, return a book, convert a book to a CSV row, and display a book as text.

The attribute available is used to specify if a book is available for borrowing by a member at this time.

2.3 Member Class

The Member class is for a registered user of the library.

Attributes include:

Member ID

Name

Email Address

Methods can be used to update the email address, verify IDs, prepare data for CSV storage, and show member information.

The Member class makes sure that information on library users is kept apart from books and loans.

2.4 Loan Class

The Loan class records borrowing transactions.

Attributes include:

Loan ID

Book ID

Member ID

Loan Date

Return Date

The class contains functions to record returned books, to check if a book is currently on loan, to prepare the data for CSV storage and to display loan information.

This class serves as the link between books and members.

2.5 Library Class

The Library class is the main management class of the system.

It maintains collections of books, members, and loans, and offers ways to:

Adding books

Adding members

Finding books

Finding members

Borrowing books

Returning books

Saving records

Loading records

The Library class oversees all significant functions of the program.

2.6 Data Management

The system uses three CSV files to store information:

books.csv

members.csv

loans.csv

The choice of CSV files was made due to their simplicity, lightness and ease of reading.

Data stored is automatically loaded when the program is run. All records are stored in the CSV files when the user logs out of the program.

This way information can be retained between program sessions.

3 Implementation Overview

3.1 Object-Oriented Programming

The entire project was implemented using OOP.

The system was broken down into classes that represent real world entities. Books, members, and loans are represented as objects, with attributes and methods that are appropriate to their respective roles.

This design helps to make the code more readable by having similar functionality grouped together.

For example, the Book class is used to perform operations on books and the Member class is used to perform operations on members.

3.2 Functions and Methods

Specific tasks were carried out in each class using methods.

Examples include:

borrow_book()

return_book()

update_email()

mark_returned()

save_all()

load_all()

Methods help to minimise duplication and organise code.

3.3 Control Structures

The project uses several control structures.

Conditional statements like if, elif, and else are used a lot to make decisions based on user input and program conditions.

Examples include:

Determining if a book is available.

Testing for the presence of a member.

Checking if a loan is active or not.

Preventing duplicate IDs.

Loop structures are also used.

When searching through lists of books, members, and loans, use for loops.

The main menu is controlled by a while loop, and the program will run until the user decides to exit.

3.4 File Handling

The csv module of Python was used for file handling.

When the user quits the program, data is saved to CSV files. There is a

method in each class that can convert the object data into a format that can be stored in CSV. At the beginning of the program, the data is loaded from the CSV files and converted back into Python objects.

This way, information will still be available even if the application is closed.

3.5 Exception Handling

Exception handling was done with try and except blocks.

The system throws ValueError exceptions if any invalid operations are performed.

Examples include:

Trying to check out a book that isn't in the library.

Trying to check out a book that is checked out.

Trying to register duplicate IDs.

Exception handling helps to make the program user friendly, by avoiding program crashes and providing meaningful error messages.

3.6 Modular Design

The program is organised into multiple Python files.

These include:

main.py

library.py

book.py

member.py

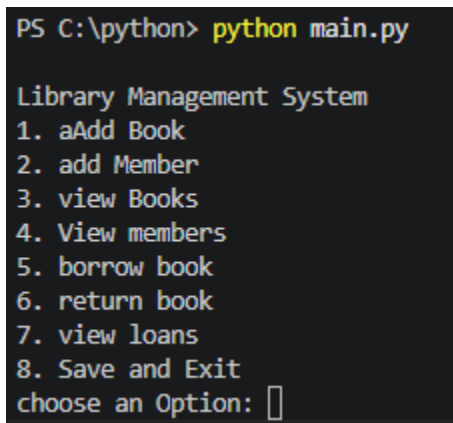
loan.py

The project is broken down into modules to make it easier to read and maintain in the future.

The responsibility for each module is clearly defined.

4 Testing

Testing was done during the development process to make sure that each feature worked properly.



```
PS C:\python> python main.py

Library Management System
1. aAdd Book
2. add Member
3. view Books
4. View members
5. borrow book
6. return book
7. view loans
8. Save and Exit
choose an Option: 
```

The first test is to add a book.

Test 1: Add a book.

Input:

Book ID: 6769

Title: The Hobbit

Author: J.R.R. Tolkien

Genre: Fantasy

Year: 1937

Expected Result:

The book needs to be added to the system.

Actual Result:

The book was successfully added and displayed when viewing books.

```
Library Management System
1. aAdd Book
2. add Member
3. view Books
4. View members
5. borrow book
6. return book
7. view loans
8. Save and Exit
choose an Option: 1
book ID: 6769
Title: The Hobbit
author: J.R.R. Tolkien
Genre: Fantasy
year: 1937
book added Successfully.
```

```
Library Management System
1. aAdd Book
2. add Member
3. view Books
4. View members
5. borrow book
6. return book
7. view loans
8. Save and Exit
choose an Option: 3
ID: 420 | Title: Gios story | author: gio | genre: thriller | Year: 2067 | status: BOrrowed
ID: 776 | Title: tales of nika | author: nika | genre: unthriller | Year: 1 | status: BOrrowed
ID: 10009 | Title: zurabas muraba | author: zuka | genre: thrilled | Year: 420 | status: BOrrowed
ID: 80082 | Title: kapelis mtexavi | author: irakla | genre: sakapelo | Year: 2 | status: BOrrowed
ID: 6769 | Title: The Hobbit | author: J.R.R. Tolkien | genre: Fantasy | Year: 1937 | status: available
```

Test 2: Add a Member

Input:

Member ID: 4201

Name: Gio

Email: Gio@Top.com

Expected Result:

The member should be successfully stored.

Actual Result:

The member was correctly displayed in the member list.

```
Library Management System
1. aAdd Book
2. add Member
3. view Books
4. View members
5. borrow book
6. return book
7. view loans
8. Save and Exit
choose an Option: 2
member id: 4201
Name: gio
email: gio@top.com
member added Successfully.
```

```
Library Management System
1. aAdd Book
2. add Member
3. view Books
4. View members
5. borrow book
6. return book
7. view loans
8. Save and Exit
choose an Option: 4
Member ID: 550 | Name: iko | Email: iko@niko.com
Member ID: 660 | Name: baska | Email: baska@q5.com
Member ID: 4201 | Name: gio | Email: gio@top.com
```

Test 3: Borrowing a Book

Input:

Book ID: 6769

Member ID: 4201

Expected Result:

A loan record should be made and the book should be unavailable.

Actual Result:

The loan was successfully recorded.

```
Library Management System
1. aAdd Book
2. add Member
3. view Books
4. View members
5. borrow book
6. return book
7. view loans
8. Save and Exit
choose an Option: 5
book ID: 6769
Member id: 4201
book Borrowed Successfully.
```

```
Library Management System
1. aAdd Book
2. add Member
3. view Books
4. View members
5. borrow book
6. return book
7. view loans
8. Save and Exit
choose an Option: 7
Loan ID: 1 | Book ID: 6769 | Member Id: 4201 | Loan Date: 2026-06-25 | Active
```

Test 4: Returning a Book

Input:

Book ID: 6769

Expected Result:

The loan should be noted as returned and the book should be made available again.

Actual Result:

The loan record has been successfully updated.

```
Library Management System
1. aAdd Book
2. add Member
3. view Books
4. View members
5. borrow book
6. return book
7. view loans
8. Save and Exit
choose an Option: 6
Book ID: 6769
Book Returned successfully.
```

```
Library Management System
1. aAdd Book
2. add Member
3. view Books
4. View members
5. borrow book
6. return book
7. view loans
8. Save and Exit
choose an Option: 7
Loan ID: 1 | Book ID: 6769 | Member Id: 4201 | Loan Date: 2026-06-25 | Returned: 2026-06-25
```

Test 5: Duplicate ID Validation.

Input:

Try to make a book using an existing ID.

Expected Result:

The system should not accept the duplicate record.

Actual Result:

The user was shown an error message saying that the ID already exists.

```
Library Management System
1. aAdd Book
2. add Member
3. view Books
4. View members
5. borrow book
6. return book
7. view loans
8. Save and Exit
choose an Option: 1
book ID: 6769
Title: blood meridian
author: tolstoy
Genre: romance
year: 401
Error: a Book with this Id Already exists.
```

Test 6: Data Persistence

Procedure:

The program was closed and reopened with data saved.

Expected Result:

All books, members and loans must be available.

Actual Result:

All records were successfully loaded from the CSV files.


```

Library Management System
1. aAdd Book
2. add Member
3. view Books
4. View members
5. borrow book
6. return book
7. view loans
8. Save and Exit
choose an Option: 8
Data saved. goodbye.
PS C:\python> python main.py

Library Management System
1. aAdd Book
2. add Member
3. view Books
4. View members
5. borrow book
6. return book
7. view loans
8. Save and Exit
choose an Option: 3
ID: 420 | Title: Gios story | author: gio | genre: thriller | Year: 2067 | status: BOrrowed
ID: 776 | Title: tales of nika | author: nika | genre: unthriller | Year: 1 | status: BOrrowed
ID: 10009 | Title: zurabas muraba | author: zuka | genre: thrilled | Year: 420 | status: BOrrowed
ID: 80082 | Title: kapelis mtexavi | author: irakla | genre: sakapelo | Year: 2 | status: BOrrowed
ID: 6769 | Title: The hobbit | author: J.R.R. Tolkien | genre: Fantasy | Year: 1937 | status: available

```

5 Reflection

This project gave hands-on experience of implementing the concepts of Python programming to solve a real-life problem. During the development process, the following concepts were used: Classes, Methods, File handling, Exception handling, Modular design.

A problem that arose during development was to make sure that information is still available when the application is closed. At first, data were only available during the program's execution. This problem was addressed by adding CSV file storage and loading capabilities.

Another difficulty was to avoid duplicate records. If not validated, users might be able to create several books or members with the same ID. To enhance data integrity, further validation techniques were thus added.

The project also emphasized the modular programming concept. The code was easier to manage and understand because the application was split into several files.

With more development time, several improvements could be made. This includes search, record editing, record deletion, enhanced input validation, and graphical user interface. These improvements would make the system more user friendly and more appropriate for larger organisations.

Overall the project was a success and the concepts of programming taught in the module were put into practice.

6 References

Works Cited

Das, Udayan, et al. "1.2 Input/Output - Introduction to Python Programming | OpenStax."

Openstax.org, 13 Mar. 2024,
openstax.org/books/introduction-python-programming/pages/1-2-input-output.

Python Software Foundation. "3.7.3 Documentation." *Python.org*, 2019, docs.python.org/3/.

van Rossum, Guido, et al. "PEP 8 – Style Guide for Python Code." *Peps.python.org*, 5 July 2001, peps.python.org/pep-0008/.